

Rail Fence

AIM

To implement the **Rail Fence Cipher encryption technique** in C to encrypt a plaintext message by arranging it in a zigzag pattern across multiple rails and then reading the characters row by row to produce the ciphertext.

Algorithm

1. **Start** the program.
2. Enter the plaintext message.
3. Enter the number of rails (key).
4. Create a matrix with the given number of rails and columns equal to the length of the plaintext.
5. Fill the matrix in a zigzag pattern:
 - Move downward until the last rail is reached.
 - Then move upward diagonally until the first rail is reached.
 - Repeat until all characters are placed.
6. Read the characters row by row from the matrix.
7. Store the characters as the ciphertext.
8. Display the ciphertext.
9. **Stop** the program.

Program :

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char text[100];
```

```
    char rail[10][100];
```

```
    int key, len, i, j;
```

```
    int row = 0, dir = 1;
```

```
    printf("Enter the plaintext: ");
```

```
scanf("%s", text);
```

```
printf("Enter the number of rails: ");
```

```
scanf("%d", &key);
```

```
len = strlen(text);
```

```
// Initialize the rail matrix
```

```
for(i = 0; i < key; i++)
```

```
{
```

```
    for(j = 0; j < len; j++)
```

```
    {
```

```
        rail[i][j] = '\n';
```

```
    }
```

```
}
```

```
// Fill the matrix in zigzag pattern
```

```
for(i = 0; i < len; i++)
```

```
{
```

```
    rail[row][i] = text[i];
```

```
    if(row == 0)
```

```
        dir = 1;
```

```
    else if(row == key - 1)
```

```
        dir = -1;
```

```
    row += dir;
```

```
}
```

```
printf("\nCiphertext: ");
```

```
// Read the matrix row by row
```

```

for(i = 0; i < key; i++)
{
    for(j = 0; j < len; j++)
    {
        if(rail[i][j] != '\n')
            printf("%c", rail[i][j]);
    }
}

return 0;
}

```

OUTPUT :

The screenshot shows a C++ IDE with two windows. The left window displays the source code for a Railfence Cipher, and the right window shows the program's output.

Source Code (Railfence Cipher.cpp):

```

1 #include <string.h>
2
3 int main()
4 {
5     char text[100];
6     char rail[100][100];
7     int key, len, i, j;
8     int row = 0, dir = 1;
9
10    printf("Enter the plaintext: ");
11    scanf("%s", text);
12
13    printf("Enter the number of rails: ");
14    scanf("%d", &key);
15
16    len = strlen(text);
17
18    // Initialize the rail matrix
19    for(i = 0; i < key; i++)
20    {
21        for(j = 0; j < len; j++)
22        {
23            rail[i][j] = '\0';
24        }
25    }
26
27    // Fill the matrix in zigzag pattern
28    for(i = 0; i < len; i++)
29    {
30        rail[row][i] = text[i];
31
32        if(row == 0)
33            dir = 1;
34        else if(row == key - 1)
35            dir = -1;
36        row += dir;
37    }
38
39    printf("Cipher text: ");
40
41    // Read the matrix row by row
42    for(i = 0; i < key; i++)
43    {
44        for(j = 0; j < len; j++)
45        {
46            if(rail[i][j] != '\0')
47                printf("%c", rail[i][j]);
48        }
49    }
50
51    return 0;
52 }

```

Output:

```

Enter the plaintext: HELLO
Enter the number of rails: 3

Cipher text: HOELL

Process exited after 5.777 seconds with return value 0
Press any key to continue . . .

```

The IDE's status bar at the bottom shows: Line: 54 Col: 2 Sel: 0 Lines: 54 Length: 999 Insert Done parsing in 0 seconds.